Intelligent Invoice Workflow Orchestration

Executive Summary

Invoice handling previously required a person to find invoice emails, open attachments, enter invoice details, compare vendor records with the MDM masters, check contracts, and maintain spreadsheets. The application connects these steps in one local workflow. It reads invoices from classic Outlook's `Invoices` folder or manual uploads, extracts document fields, validates them, resolves a Billing Account from the Vendor Accounts master, checks the linked contract, and writes cumulative CSV and Excel reports. Finance and operations staff can review exceptions instead of re-entering every invoice. The application is a working local prototype; production accuracy, time savings, and corporate approval still need to be measured and confirmed.

Business Problem

The previous process involved several manual handoffs. An operator located invoice messages, downloaded PDFs, read the invoice, copied values to a tracking sheet, searched for the vendor in MDM, chose the appropriate account, checked the contract dates, and updated downstream files. At higher email volumes, repeated data entry increases turnaround time and the chance of transcription errors, missed attachments, duplicate invoices, or selection of the wrong billing account. The people affected are invoice processors, MDM or contract reviewers, and the teams that use the resulting reports.

The business need is to orchestrate the whole sequence while keeping ambiguous or incomplete invoices visible for human review. Automation should not silently guess an account or imply that an unverified extraction is accurate.

Proposed Solution

The application provides a local invoice-processing pipeline. A user can upload PDF, image, ZIP, Outlook `.msg` or `.eml`, CSV, or XLSX files; on Windows, the **Run Outlook** action reads actual attachments from the signed-in classic Outlook mailbox's `Invoices` folder. A per-user Windows scheduled task can repeat Outlook intake every 30 minutes while the user is signed in and Outlook is running. Existing messages and invoice results are checked for duplicates.

Native PDF text or Tesseract OCR supplies a locally trained Python field-selection model and parsers. The application extracts invoice identity, dates, currency, amounts, service lines, vendor location, and available contact information. Validation checks required fields and amount or line-item consistency. Vendor-name normalization identifies candidate MDM rows; when multiple accounts belong to one vendor, the invoice or explicitly labelled email region is compared with `Restricted To`. The chosen Billing Account is written to the invoice result, not to the Vendor Accounts master. The matching contract is then checked for one record with usable Start Date and End Date. Unmatched or ambiguous cases retain a blank account and an explicit review reason.

Each batch updates durable local history and cumulative CSV/Excel outputs. A separate contract-verified export contains only successful invoices with a uniquely matched account and a valid contract date record. Operators can correct a result and save reviewed examples for incremental local training. Review is still required for incomplete, inconsistent, duplicate, or unresolved records.

Key Features

1. Intake from manual uploads and the classic Outlook `Invoices` folder, with attachment-only extraction, duplicate skipping, and processed messages marked read.
2. Local PDF text extraction, image OCR, field selection, service-line parsing, and visible validation or review reasons.
3. Vendor normalization and MDM Billing Account resolution, with region used to distinguish multiple accounts and no arbitrary selection when a tie remains.
4. Contract date-presence verification, human correction, audit history, and two cumulative CSV feeds plus an Excel workbook.

Architecture

Detailed architecture diagram: docs/architecture/solution-architecture.md

```
Manual upload or Outlook Invoices folder
            |
            v
Attachment and archive intake -> PDF text or Tesseract OCR
            |
            v
Local field selection and parsers -> Validation and duplicate checks
            |
            v
Vendor Accounts name and region match -> Contracts date check
            |
            v
Review or approved result -> SQLite history -> CSV and Excel reports
```

The React interface calls the local FastAPI service. `services/document_processing.py` coordinates extraction; `services/vendor_matching.py` resolves MDM accounts; `services/local_storage.py` maintains history; `services/mdm_csv_service.py` builds the MDM exports. The Vendor Accounts and Contracts CSV files are read-only reference masters for this workflow. The local storage folder contains invoice originals, reviewed examples, model artifacts, history, and outputs.

Technology Stack

Layer	Implemented technology
Frontend	React 19, FLOWSTACK UI Brick 0.2.2, Vite 7
Backend	Python, FastAPI, Uvicorn, Pydantic
Durable storage	SQLite for saved history; local CSV and XLSX exports; local source archive
Document extraction	PyMuPDF for PDF text, Tesseract with pytesseract for OCR, Pillow for images, extract-msg for Outlook files
Local ML	scikit-learn and joblib for a supervised field-selection model; reviewed corrections support local retraining
Integration	pywin32 with classic Outlook on Windows; local Vendor Accounts and Contracts CSV masters
Deployment	Local Windows process on `127.0.0.1`; optional per-user Windows Task Scheduler job

Layer	Implemented technology
Testing	pytest tests and a Vite production build

No Gemini, Ollama, Microsoft Graph, Power Automate, cloud inference API, or inbound email service is required for this local configuration.

Setup and Execution

On Windows, install Python, Node.js with npm, and Tesseract OCR through approved company channels. Open PowerShell in the `invoice_ai` project directory and run:

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
cd frontend
npm ci
npm run build
cd ..
.\.venv\Scripts\python.exe -m uvicorn web_app:app --host 127.0.0.1 --port 8502
```

Open `http://127.0.0.1:8502/` on the same computer. If Tesseract is not found, copy `.env.example` to `.env` and set `TESSERACT_CMD` to the approved installation path. Keep storage on an access-controlled local drive; configure `LOCAL_STORAGE_DIR` before first use if the default location is unsuitable. Do not copy private invoice history or model artifacts into a public repository.

For Outlook intake, use classic Outlook under the same Windows account as the app, create or identify a top-level `Invoices` folder in one mailbox, and select **Run Outlook**. Set `OUTLOOK_MAILBOX` if more than one mailbox contains such a folder. The optional `scripts/install_outlook_schedule.ps1` installs a per-user 30-minute task; it requires Outlook to stay open and the user to be signed in. Company policy may block programmatic Outlook access; manual upload remains available.

For tests, install the development requirements and run:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-dev.txt
.\.venv\Scripts\python.exe -m pytest -q
cd frontend
npm run build
```

The latest full-suite result and limitations should be confirmed on the submission computer before declaring the application ready. `README.md` contains operational details and current extraction limitations.

Demo

- Demo video: Not yet provided.
- Presentation: Not yet provided.
- Application URL: `http://127.0.0.1:8502/` for a local demonstration only; this is not a public URL.
- Suggested demonstration: run an approved synthetic invoice through upload or Outlook intake, inspect extracted fields and review reasons, run MDM matching, then download both cumulative reports. Use synthetic data only in a shared recording.

Business Benefits

- **Time saved:** The pipeline removes repeated downloading, reading, re-keying, and file consolidation for invoices it can process. No measured minutes or annual saving are claimed yet.

- **Cost reduction:** Less manual handling may reduce processing effort, subject to a timed pilot and approval of the local support model.
- **Risk reduction:** Duplicate checks, arithmetic validation, MDM ambiguity flags, contract date checks, and traceable review reasons help expose issues before a result is treated as complete.
- **User experience:** One intake and review screen replaces several manual handoffs; two cumulative exports support downstream work.
- **Measurement plan:** Record baseline manual handling time, automated processing time, human correction time, field-level accuracy on real approved invoices, duplicate detection, and unresolved MDM cases over a representative pilot.

Responsible AI and Security

The current extractor runs locally and sends no invoice content to a remote model service. The web server is intended for loopback access. Source files, OCR text, model artifacts, reviewed corrections, and SQLite history may all contain confidential information and require corporate-approved storage, access controls, retention, and backups. The app does not replace payment approval or a human check of critical fields. Review reasons expose missing or inconsistent fields, duplicate suspicions, and unresolved account or contract matches. Synthetic examples should be used for demos and repository submissions; no API keys, customer invoices, personal data, or credentials should be committed.

Limitations and Assumptions

- The local model is a supervised document extractor, not a newly trained general-purpose LLM. OCR quality, invoice layout, and training coverage affect results. A rule-validated result is not an independently measured accuracy score; the 90 percent target remains a goal to verify on representative real invoices.
- Business Unit and Owner remain blank without an authoritative mapping. Billing Account stays blank when the MDM vendor or region match is unresolved. Missing service dates remain blank; the app does not manufacture them.
- Contract verification checks that a uniquely matched contract record has usable Start Date and End Date. It does **not** currently require the invoice service period to fall within the contract period.
- Outlook automation needs classic Outlook, the local Windows user session, and permission for the Outlook object model. The scheduled task processes batches rather than guaranteeing immediate receipt.
- MDM masters are read, not rewritten. The second cumulative CSV is an export for downstream use; `Insert` is a marker, not proof of insertion into another corporate system.
- Approval of open-source dependencies, deployment, retention, and use with company invoices remains with the organization.

Future Scope

Run a controlled pilot with approved real invoices and a labelled evaluation set. Use the findings to improve OCR and field extraction, calibrate confidence, document required fields, and set a review threshold based on measured field-level accuracy. Add governed Business Unit and Owner mappings; consider an approved enterprise mail integration only if IT permits it. Add operational monitoring, backup and retention policies, role-based access, and a controlled downstream import after business validation.

Open Source and Third Party Inventory

The direct application dependencies are specified in `requirements.txt` and `frontend/package.json`; exact installed versions are resolved from the Python environment and `frontend/package-lock.json`. Obtain packages through approved company registries and verify each license and version before enterprise deployment. This table is an inventory, not an approval record.

Component	Declared version	Purpose	Source or registry	Approval and license status
pydantic	>=2.0,<3	Data validation	Python package registry	Company review required
python-dotenv	>=1.0,<2	Local configuration	Python package registry	Company review required
PyMuPDF	>=1.24,<2	PDF text access	Python package registry	Company review required
pytesseract	>=0.3.13,<1	OCR bridge	Python package registry	Company review required
Pillow	>=10.4,<12	Image handling	Python package registry	Company review required
extract-msg	>=0.55,<0.57	Outlook file parsing	Python package registry	Company review required
openpyxl	>=3.1.5,<4	Excel export and import	Python package registry	Company review required
fastapi	>=0.115,<1	HTTP API	Python package registry	Company review required
uvicorn	>=0.30,<1	Local web server	Python package registry	Company review required
python-multipart	>=0.0.9,<1	Multipart uploads	Python package registry	Company review required
scikit-learn	>=1.7,<2	Local supervised model	Python package registry	Company review required
joblib	>=1.5,<2	Model persistence	Python package registry	Company review required
pywin32	312 on Windows	Outlook desktop access	Python package registry	Company review required
React and React DOM	19.2.0	User interface	npm registry	Company review required
FLOWSTACK UI Brick	0.2.2	Interface components	npm registry	Company review required
Vite	7.1.7	Frontend build	npm registry	Company review required
Tesseract OCR	Installed separately	OCR executable	Approved installer	Version and license review required

Final Submission Checklist

- [ ] Start the app on the submission computer and run the full Python suite plus frontend build.
- [ ] Confirm the repository includes current source, setup instructions, and this architecture description.
- [ ] Add accessible demo video and presentation links if required by the submission.
- [ ] Confirm no secrets, private invoices, personal data, or local history are included in the repository or demo.
- [ ] Obtain company approval for dependencies, installation, Outlook access, and invoice data handling.
- [ ] Record a representative field-level accuracy and time-saved pilot before stating quantified benefits.